

Multi-Domain Recommendation Portal

AI Lab Project - Complete Project Plan

Movies | Products | Study and Career | Diet

1. Project Overview

A web portal where a user picks a category, enters a few preferences, and receives personalised Top-N recommendations with a short explanation of *why* each item was suggested. Each category uses its own model behind one shared interface, so the portal can be extended easily.

Item	Details
Goal	Build and evaluate a working multi-category recommender portal with a demo-ready UI
Core categories	Movies, Products, Study/Career (must have)
Bonus category	Diet (meal suggestions by preference and calorie goal - not medical advice)
Approach	Content-based filtering, collaborative filtering, then a hybrid
Duration	6 weeks (adjust to your deadline)
Deliverables	Working portal, source code, evaluation results, report, presentation, demo video

2. Scope and Priorities

Category	Dataset options	Method	Difficulty	Priority
Movie	MovieLens, TMDB	TF-IDF + cosine, SVD / KNN	Easy	Must
Product	Amazon reviews, Kaggle e-commerce	Content-based + item KNN	Easy-Medium	Must
Study / Career	Kaggle courses / jobs data, or a self-made survey	Skill-interest matching + cosine	Medium	Must
Diet	Recipe and nutrition datasets (Kaggle)	Filter by calories / veg / allergy + similarity	Hard	Bonus

Rule: finish one category fully (data to model to UI) before starting the next. Diet is added only if time remains, and always carries a visible disclaimer.

3. Features

Must-have

- User sign-up, login and a saved profile
- Home page with a card or tab for each category
- Category-specific preference input forms
- Recommendation engine returning Top-N results per category
- Results page with item cards, description and match score
- "Why recommended" explanation for every result

- Like / dislike or star rating saved to the database

Good-to-have

- Hybrid model (content-based + collaborative)
- Cold-start handling for new users (popular items and onboarding questions)
- Search and filters (genre, price, rating, etc.)
- Saved items / history page
- Analytics dashboard (model metrics, user activity)
- Natural-language input or chatbot (e.g. "a sci-fi movie like Interstellar")
- Diet filters: vegetarian / vegan / allergies, plus a medical disclaimer

4. Tech Stack and Architecture

Layer	Tools
Language	Python 3
Data and ML	Pandas, NumPy, scikit-learn, Surprise (or implicit)
Web / UI	Streamlit (fastest) or Flask + HTML/CSS
Database	SQLite (users, ratings, saved items)
Visualisation	Matplotlib / Plotly
Version control	Git + GitHub

Architecture flow

User UI login, inputs	Backend app.py router	Category model movie / product / career / diet	Results Top-N + explanation	Database feedback saved
---------------------------------	---------------------------------	---	---------------------------------------	-----------------------------------

Every category module exposes the same function, `recommend(user_input)` -> `List`, so the UI treats all of them identically.

Suggested folder structure

```

recommender-portal/
  data/                raw and cleaned datasets
  models/
    movie_rec.py
    product_rec.py
    career_rec.py
    diet_rec.py
  app.py              Streamlit / Flask entry point
  database.py          users, ratings, saved items
  evaluation.py        precision@k, recall@k, RMSE
  requirements.txt
  README.md

```

5. Week-by-Week Plan

Week	Focus	Tasks	Output
1	Plan and setup	Finalise scope and categories; pick datasets; set up GitHub repo and environment; split tasks in the team; read 2-3 related papers or articles	Scope document, repo, datasets downloaded
2	Data and first model	Clean and explore data; build the movie recommender (TF-IDF + cosine); add SVD / KNN; record metrics	Movie notebook + saved model + metrics
3	Remaining models	Build the product and study/career recommenders using the same template; start the diet module if time allows; add explanation text logic	3 working models, unified recommend() interface
4	Portal build	Create the UI with one page per category; connect models; add login, SQLite database and the rating/feedback system	Working end-to-end portal
5	Polish and testing	Add filters, saved items, dashboard and cold-start handling; test with 5-10 classmates; fix bugs; improve the UI	Stable portal + user feedback summary
6	Report and delivery	Write report; draw architecture diagram; record demo video; prepare slides; rehearse the live demo	Final report, PPT, demo video, final code

6. Suggested Task Split (for a team of 3-4)

Role	Responsibilities
Data and models lead	Dataset cleaning, movie and product recommenders, evaluation metrics
Models and logic	Study/career and diet recommenders, explanation generation, cold-start logic
Frontend / backend	Streamlit or Flask UI, login, database, connecting models to pages
Documentation and testing	Report, slides, demo video, user testing, README, bug tracking

Working solo? Follow the same order and drop the diet module and chatbot.

7. Evaluation Plan

- **Data split:** 80% train / 20% test (or per-user holdout for ratings data)
- **Metrics:** Precision@K, Recall@K and RMSE (for rating prediction); compare content-based vs collaborative vs hybrid
- **Baseline:** compare against a simple popularity-based recommender to show your model adds value
- **User testing:** 5-10 classmates rate the usefulness of results on a 1-5 scale
- **Reporting:** include a results table and charts in the report and dashboard

8. Risks and Mitigation

Risk	Mitigation
Too many categories, not enough time	Lock 3 core categories; treat diet as a bonus
Poor or messy data	Choose well-known datasets early; allocate time in Week 2 for cleaning
Cold-start (new users, no history)	Show popular items and ask 3-5 onboarding questions
Diet seen as medical advice	Use only preference/calorie-based suggestions; add disclaimer and allergy filters
UI not ready for the demo	Start the UI by Week 3; record a backup demo video

9. Final Deliverables Checklist

- Working portal (live demo + hosted or local run instructions)
- Source code on GitHub with a clear README
- Evaluation results with metrics tables and charts
- Project report: problem statement, related work, architecture, methodology, results, limitations, future scope
- Presentation slides and a backup demo video

10. Future Scope

- Deep-learning recommenders (neural collaborative filtering)
- LLM-powered conversational recommendations
- Real-time learning from user feedback
- Mobile-friendly version and cloud deployment